
Author

author: name: Мохаммед Фуад Мохаммед Хамуд Аль-хадри
degrees: BSc orcid: N/A email: 1132255653@rudn.ru affilia-
tion: - name: Российский университет дружбы народов country:
Российская Федерация postal-code: N/A city: Москва address: ул.
Миклухо-Маклая, д.21 к1

Title

title: "Отчёт по лабораторной работе №4" subtitle: "Продвинутое
использование Git: Gitflow и семантические коммиты" license:
"CC BY" -----

Цель работы

Целью лабораторной работы является освоение продвинутого
рабочего процесса с Git, включая использование Gitflow,
семантического версионирования и Conventional Commits.

Задание

- Установить Gitflow и Node.js.
- Настроить репозиторий с Gitflow и Conventional Commits.
- Создать функциональные ветки и релизные ветки.
- Создать журнал изменений (CHANGELOG).
- Выполнить публикацию на GitHub с релизами.

Теоретическое введение

Gitflow Workflow — это модель ветвления для управления
проектами с учётом релизов:

- Ветка master — официальная история релиза.
- Ветка develop — объединение всех функций.
- Ветки feature — разработка отдельных функций.
- Ветки release — подготовка релизов.
- Ветки hotfix — исправления ошибок в master.

Семантическое версионирование (SemVer):

- Формат: MAJOR.MINOR.PATCH

- MAJOR — несовместимые изменения
- MINOR — новые функции без нарушения совместимости
- PATCH — исправление ошибок

Conventional Commits:

- Стандартизированная структура сообщений коммитов:

<type>(<scope>): <subject>

[optional body]

[optional footer]

- Основные типы: fix, feat, chore, docs, style, refactor, perf, test

Выполнение лабораторной работы

Установка программного обеспечения

Установка Gitflow:

```
dnf copr enable elegos/gitflow
dnf install gitflow
```

Установка Node.js и PNPM:

```
dnf install nodejs
dnf install pnpm
```

Настройка PNPM:

```
pnpm setup
source ~/.bashrc
```

Настройка Conventional Commits

Установка commitizen и standard-changelog:

```
pnpm add -g commitizen
pnpm add -g standard-changelog
```

Настройка package.json для commitizen:

```
"config": {
  "commitizen": {
    "path": "cz-conventional-changelog"
```

```
}  
}
```

Инициализация репозитория Git и Gitflow

Создание репозитория и первая фиксация:

```
git init  
git commit -m "first commit"  
git remote add origin git@github.com:<mod11010001>/git-extended.git  
git push -u origin master
```

Инициализация Gitflow:

```
git flow init
```

Работа с ветками

Функциональные ветки

Создание ветки функции:

```
git flow feature start feature_branch
```

Завершение работы:

```
git flow feature finish feature_branch
```

Релизные ветки

Создание релиза:

```
git flow release start 1.0.0  
standard-changelog --first-release  
git add CHANGELOG.md  
git commit -am 'chore(site): add changelog'  
git flow release finish 1.0.0  
git push --all  
git push --tags  
gh release create v1.0.0 -F CHANGELOG.md
```

Ветки исправлений (hotfix)

```
git flow hotfix start hotfix_branch  
git flow hotfix finish hotfix_branch
```

```
git push --all
git push --tags
```

Результаты

- Репозиторий переведён на Gitflow
 - Настроено семантическое версионирование
 - Созданы функциональные, релизные и hotfix ветки
 - Создан журнал изменений (CHANGELOG.md)
 - Выполнены публикации на GitHub с релизами
-

Выводы

В лабораторной работе были изучены продвинутые возможности Git:

- модель ветвления Gitflow
- семантическое версионирование
- Conventional Commits
- автоматизация релизов и журналов изменений

Освоение этих навыков позволяет организовать профессиональный процесс разработки ПО с качественным управлением версиями.

Список литературы

::: {#refs}

1. Vincent Driessen, "A successful Git branching model", 2010
2. Conventional Commits Specification – <https://www.conventionalcommits.org>
3. Standard Changelog – <https://github.com/conventional-changelog/standard-version>